

Logic Vulnerabilities: Input Validation and IDOR

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWL UW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Stefano Alberto (stefano.alberto@polito.it)

v. 1.0.0 (6/2/2026)

Contents

1	Introduction	2
2	Exercises	4
3	Extras	10

1 Introduction

The purpose of this laboratory is to introduce the identification and exploitation of logic vulnerabilities in web applications. These vulnerabilities do not arise from technical flaws such as injection or parsing errors, but from mistakes in the application's business logic and input handling.

While the vulnerabilities themselves are conceptually simple, the main challenge lies in learning how to use the right tools to discover and exploit them effectively.

Attack surface

When attacking (or developing) a web application, the first step is to identify the **attack surface**: all the points where user-controlled data enters the application.

The most obvious inputs are form fields (e.g. search boxes, login forms), but as attackers, we must consider the **entire HTTP request** as potential input. This includes:

- **URL parameters** (query string and path),
- **Request body** (POST data, JSON payloads),
- **Cookies**,
- **HTTP headers** (Referer, User-Agent, custom headers),

All of these can be controlled by an attacker and must be properly validated on the server side. A common mistake is to trust data simply because it cannot be edited directly through the normal user interface.

Burp Suite

To intercept and manipulate HTTP requests beyond what a browser normally allows, we use **Burp Suite** (Community Edition). Burp acts as a proxy between the browser and the web server, letting you:

- Inspect every request and response,
- Modify any part of a request before it reaches the server,
- Replay and automate requests.

Make sure Burp Suite is configured as a proxy in your browser before starting the exercises. If you have not used Burp before, refer to the official getting started guide:

<https://portswigger.net/burp/documentation/desktop/getting-started>

Logic vulnerabilities

For this first lab, we focus on **logic vulnerabilities**: flaws in which the application fails to correctly enforce its own rules. Despite appearing simple, these vulnerabilities are widespread and can have a severe impact. They typically arise when developers trust client-side data that should never be trusted.

Common categories include:

- **Untrusted data**: the server uses values provided by the client (prices, scores, user roles) without verifying them.

- **IDOR (Insecure Direct Object Reference):** the application exposes internal object identifiers (e.g. sequential IDs) and fails to check whether the requesting user is authorized to access them.

For a broader overview and additional examples of business logic vulnerabilities, see:

<https://portswigger.net/web-security/logic-flaws>

<https://portswigger.net/web-security/access-control/idor>

2 Exercises

This section includes six exercises on logic vulnerabilities in web applications. The first two are solved step by step; the next two are the remaining core exercises; the last two are optional and cover more advanced concepts.

Exercise 1 – Flags Shop

Purpose:	Buy the flag from the online shop, even though you cannot afford it.
Suggested tools:	Burp Suite, Browser DevTools.
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-43

This is an online shop where the flag is for sale, but it costs too much. Your task is to find a way to purchase it anyway. Follow the steps below.

Explore the shop

Open the challenge page in your browser.

What items are for sale? What are their prices? What is your current balance?

→

Try to buy

Try to buy the *Bandiera anonymous* by clicking the buy button.

What does the server respond?

→

Response

```
1 HTTP/1.1 200 OK
2 Content-Length: 21
3 Content-Type: text/html; charset=UTF-8
4 Date: Fri, 20 Mar 2026 12:32:05 GMT
5 Server: Apache/2.4.38 (Debian)
6 X-Powered-By: PHP/7.2.34
7
8 CREDITO INSUFFICIENTE
```

Inspect the HTML source

Right-click on the page and select “View Page Source”, or open the browser DevTools (F12) and look at the Elements tab.

Look at the `<form>` element that sends the purchase request. What parameters are included in the form? What are their names and values?

→

```
1 POST /buy.php HTTP/1.1
2 Host: shops.challs.olicyber.it
3 Content-Length: 15
4 Cache-Control: max-age=0
5 Accept-Language: it-IT,it;q=0.9
6 Origin: http://shops.challs.olicyber.it
7 Content-Type: application/x-www-form-urlencoded
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/142.0.0.0 Safari/537.36
10 Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,ima
e/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://shops.challs.olicyber.it/
12 Accept-Encoding: gzip, deflate, br
13 Connection: keep-alive
14
15 id=26&pto=1000
```

NOTA

The price (`costo`) is sent by the client inside a hidden `<input>` field. Hidden fields are invisible in the browser UI, but they are **not secret**: they are part of the HTML source and can be freely read and modified by anyone. The server should **never** trust a value just because the user interface does not expose it.

Intercept with Burp Suite

Configure Burp Suite as a proxy, enable interception, and click the buy button for the Anonymous flag again. Look at the intercepted POST request. What parameters are in the request body?

→

Exploit

While the request is intercepted in Burp Suite, modify the `costo` value to something you can afford. Forward the modified request. What does the server respond now?

→

Reflection

Why is this application vulnerable? What should the developer do differently?

→

Exercise 2 – Click me

Purpose:	Win the clicking game without actually clicking enough times.
Suggested tools:	Burp Suite (Repeater), Browser DevTools (Application/Storage tab, Console).
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-46

This is a simple clicking game. Your task is to obtain the flag without clicking millions of times. Follow the steps below.

Explore the game

Open the challenge page in your browser and read the challenge description on the OliCyber platform.

What is the goal of the game? How many cookies do you need to collect? Is it feasible to do it by clicking manually?

→

Read the JavaScript

Open the browser DevTools (F12) and look at the page source. You can find the JavaScript code in the Elements tab (inside a `<script>` tag) or in the Sources tab.

How is the score tracked? Where is the value stored each time you click?

→

NOTA

The score is stored in a **client-side cookie**. Remember that cookies are sent automatically with every HTTP request to the same domain via the `Cookie` header. This means the server reads the score from the cookie value in the request — and we can control what value the server sees.

Intercept with Burp Suite

Click the cookie image a few times to generate some score, then **refresh the page**. Make sure Burp Suite is intercepting requests.

Look at the intercepted GET request. What do you see in the `Cookie` header?

→

Use Repeater

In Burp Suite, right-click the intercepted request and select “Send to Repeater”. Open the Repeater tab.

Modify the `cookies` value in the `Cookie` header to a number above 10 000 000 (e.g. `cookies=99999999`) and click “Send”.

What does the server respond?

→

NOTA

The flag check happens **server-side when the page is loaded**. The server reads the cookie value from the incoming request and checks if it exceeds the threshold. This is why refreshing (or replaying the request in Repeater) triggers the check.

Alternative approach: could you do this without Burp?

Think about where the cookie is stored. What browser tools allow you to view and edit cookies?

→

Open the DevTools, go to the **Application** tab (Chrome) or **Storage** tab (Firefox), then click on **Cookies**.

Find the `cookies` cookie, change its value to a number above 10 000 000, and refresh the page. Does it work?

→

Thinking further: script tampering

Look at the JavaScript code again. The variable `num` is incremented by 1 on each click.

Could you modify the script so that a single click is enough? How would you do it? Think of at least two different ways.

→

NOTA

There are often **multiple ways** to exploit a vulnerability. Modifying the cookie (via Burp Repeater or DevTools) is the most direct approach, but tampering with JavaScript via the Console or intercepting and rewriting the server's response are equally valid techniques. As attackers, we should always consider all the vectors available to us.

Exercise 3 – Insecure Direct Object References (IDOR)

Purpose:	Access another patient’s confidential lab results by exploiting an IDOR vulnerability.
Suggested tools:	Burp Suite, Browser.
Hints:	Observe the requests made by the application. Are there any predictable identifiers that reference objects you should not have access to?
Source:	MedLab Portal – https://offsec.m0lecon.it/challenges#LabResults-118

MedLab Portal is a clinic patient portal where users register. The application uses sequential or predictable identifiers to reference user objects, and it does not verify that the requesting user is authorized to access them.

Check out the application and use Burp Suite to intercept and modify the relevant requests.

→

Exercise 4 – Password changer 3000

Purpose:	Change the admin’s password even though the application prevents it.
Suggested tools:	Burp Suite
Hints:	Look at the flow implementing the password change. The application blocks “admin” directly. Is there a way to bypass this check?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-59

This application lets you change any user’s password except the admin’s. However, the flow consists of multiple steps. Write down the flow that implements the password change.

→

What is the `token` parameter? Can you decode it? Can you manipulate it?

→

Exercise 5 – Make a wish

Purpose:	Bypass an "impossible" validation check.
Suggested tools:	Burp Suite.
Hints:	The server-side check compares your input against a condition that seems impossible to satisfy with a normal string. What happens if you send a different <i>type</i> of data instead?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-47

This challenge has a validation check that appears impossible to bypass when sending a regular string value. The key insight is that many web frameworks (PHP in particular) allow you to send different types through request parameters.

What types of data can you send as a HTTP GET parameter?

→

Exercise 6 – EasyNotes

Purpose:	Read another user's private notes by exploiting an IDOR vulnerability in the API.
Suggested tools:	Burp Suite, Browser DevTools.
Hints:	Create a note and observe the ID assigned to it. Are note IDs unique per user or shared across all users? Did you checked carefully?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-225

This application lets users create and view personal notes. What are the IDs you use to access you notes? How does the web application works under the hoods? What HTTP requests does your browser do?

What happens if you try to access a note you didn't create?

→

3 Extras

This section lists additional challenges and resources that are not mandatory for this laboratory. They provide further practice on the topics covered in this lab and can be used for self-study.

Additional challenges

The following challenges cover similar concepts to the exercises above and can serve as extra practice.

Bibbopedia 1

A very simple warm-up challenge: a button on the page is disabled via HTML. Find a way to click it.

<https://training.olicityber.it/challenges#challenge-278>

NOTA

This challenge requires only the Browser DevTools (Inspector). It can serve as a first introduction to inspecting and modifying client-side HTML.

SoundOfSilence

A type juggling challenge similar to Exercise 5 (Make a wish).

<https://training.olicityber.it/challenges#challenge-132>

PortSwigger Web Security Academy

PortSwigger provides an excellent set of free, guided labs on many web security topics. The following learning paths are directly relevant to this laboratory:

- **Business logic vulnerabilities:** <https://portswigger.net/web-security/logic-flaws>
- **Access control and IDOR:** <https://portswigger.net/web-security/access-control>